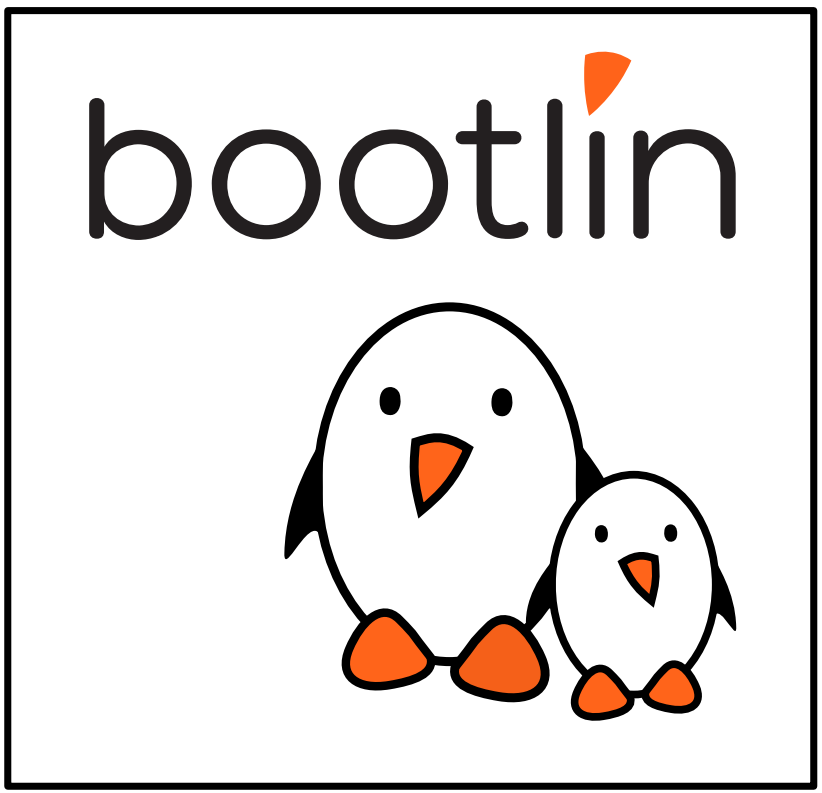
Optimizing init scripts and system startup



Methodology

There are multiple ways to reduce the time spent in init scripts before starting the application:

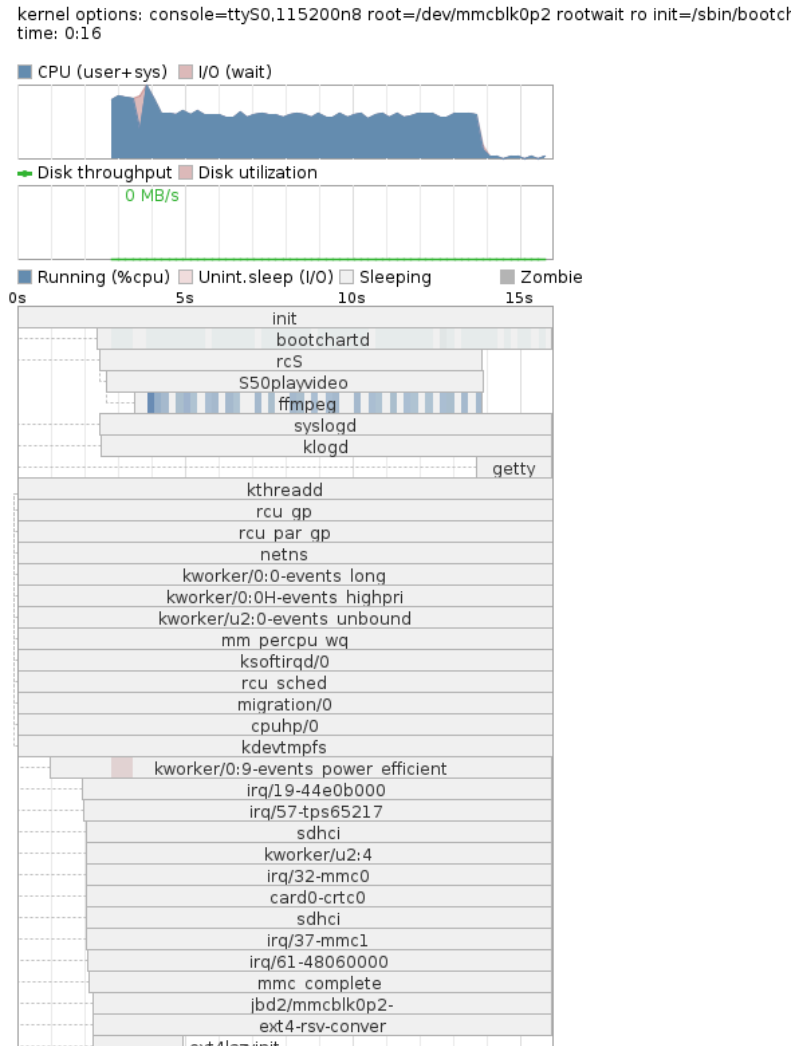
- Start the application as soon as possible after only the strictly necessary dependencies.
- Simplify shell scripts
- Even starting the application before `init`

Measuring - bootchart

- If you want to have a more detailed look at the userland boot sequence than with `grabserial`.
- You can trace processes running at init time with `bootchartd` from `busybox` (`CONFIG_BOOTCHARTD=y`)
- Boot your board passing `init=/sbin/bootchartd` on your kernel command line
- Copy `/var/log/bootlog.tgz` to your host.
- Use Bootchart from `https://bootlin.com/pub/source/bootchart-0.9.tar.bz2` (Bootchart is no longer maintained), to generate a timechart:

```
cd bootchart-<version>
java -jar bootchart.jar bootlog.tgz
```

- This produces a `bootlog.png` image



Measuring - systemd

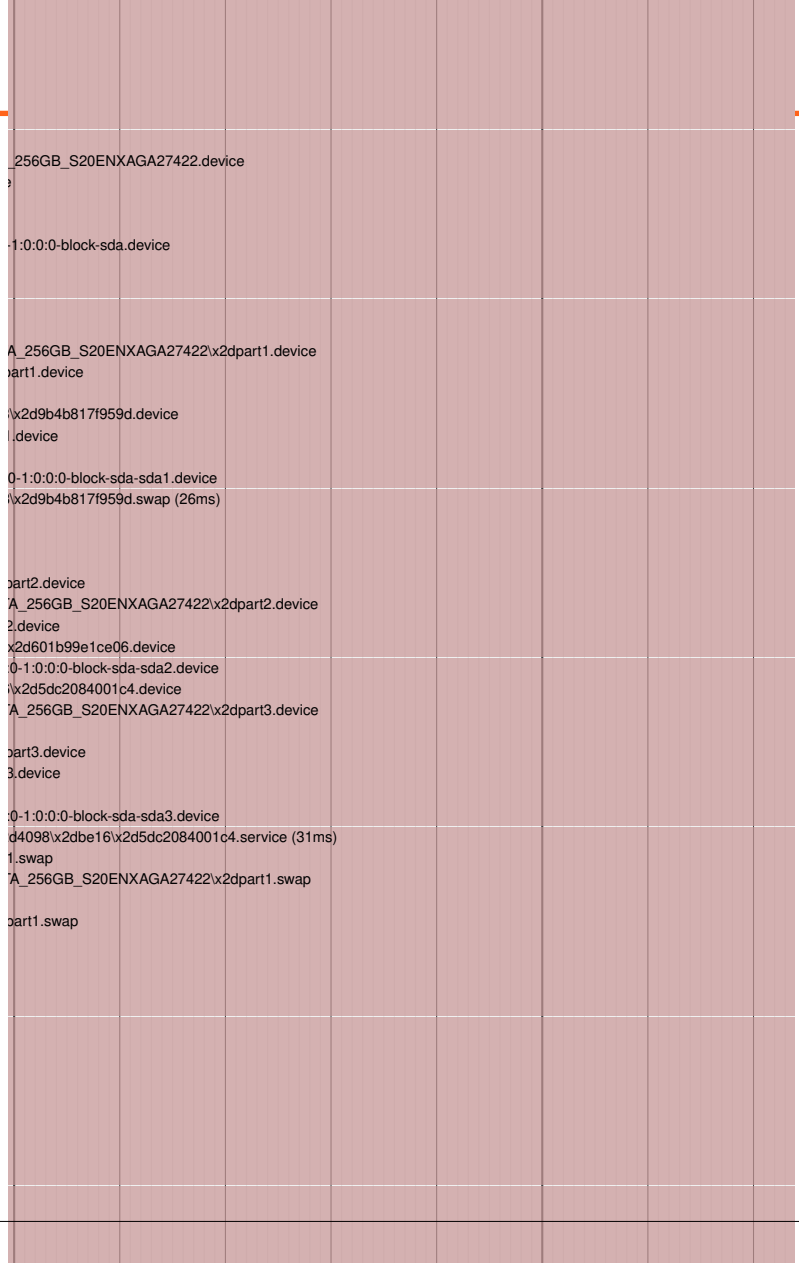
If you are using `systemd` as your `init` program, you can use `systemd-analyze`. See `https://www.freedesktop.org/software/systemd/man/systemd-analyze.html`.

```
$ systemd-analyze critical-chain multi-user.target @47.820s
└─pmie.service @35.968s +548ms
  └─pmcd.service @33.715s +2.247s
    └─network-online.target @33.712s
      └─systemd-networkd-wait-online.service @12.804s +20.905s
        └─systemd-networkd.service @11.109s +1.690s
          └─systemd-udevd.service @9.201s +1.904s
            └─systemd-tmpfiles-setup-dev.service @7.306s +1.776s
              └─kmod-static-nodes.service @6.976s +177ms
                └─systemd-journald.socket
                  └─system.slice
                    └─-.slice
```

systemd-analyze plot

This command prints an SVG graphic detailing which system services have been started at what time, highlighting the time they spent on initialization.

```
$ systemd-analyze plot >bootup.svg
$ inkscape bootup.svg
```



Init optimizations

Goal to start your application as soon as possible after all the dependencies are started:

- Depends on your `init` program. Here we are assuming BusyBox `init` scripts.
- `init` scripts run in alphanumeric order and start with a letter (K for stop (**k**ill) and S for **s**tart).
- You want to use the lowest number you can for your application.
- You can even replace `init` with your application! However, that's easier to keep a standard `init`, which also acts as a universal parent to orphan processes (otherwise you get zombies), and also takes care of implementing system shutdown.

Optimizing init scripts

- Start all your services directly from a single startup script (e.g. `/etc/init.d/rcS`). This eliminates multiple calls to `/bin/sh`.
- An easier to maintain solution allowing to keep subscripts: `source` them (`.` command) if possible. This won't spawn new shell processes. Buildroot's `/etc/init.d/rcS` file already does this with `.sh` files.
- You could mount your filesystems directly in the C code of your application:

```c
#include <stdio.h>
#include <sys/mount.h>

int main (void)
{
        int ret;
        ret = mount("sysfs", "/tmp/test", "sysfs", 0, NULL);
        if(ret < 0)
                perror("Can't mount sysfsn");
}
```

Reduce forking (1)

- `fork`/`exec` system calls are very expensive. Because of this, calls to executables from shells are slow.
- Try to use shell built-ins whenever possible. For example in BusyBox, you can use `echo`, `test`, `printf` and others as shell built-ins. At run time, use the `type` command to check whether a command is a built-in. Example: `type echo`.
- BusyBox also has a *exec prefer applets* setting (`CONFIG_FEATURE_PREFER_APPLETS`) trying to run the corresponding applet (instead of making an `exec` call), typically in shells or in commands such as `find -exec`.

Reduce forking (2)

Pipes and back-quotes are also implemented by `fork`/`exec`. You can reduce their usage in scripts. Example:

```
cat /proc/cpuinfo | grep model
```

Replace it with:

```
grep model /proc/cpuinfo
```

See https://elinux.org/Optimize_RC_Scripts

Reduce forking (3)

Replaced:

```
if [ $(expr match "$(cat /proc/cmdline)" '.* debug.*')
       -ne 0 -o -f /root/debug ]; then DEBUG=1
```

By a much cheaper command running only one process:

```
res=`grep " debug" /proc/cmdline`
if [ "\$res" -o -f /root/debug ]; then DEBUG=1
```

This only optimization allowed to save 87 ms on an ARM AT91SAM9263 system (200 MHz)!

Reduce size

- Strip your executables and libraries, removing ELF sections only needed for development and debugging. The `strip` command is provided by your cross-compiling toolchain. That's done by default in Buildroot.
- `superstrip`: `https://muppetlabs.com/~breadbox/software/elfkickers.html`. Goes beyond `strip` and can strip out a few more bits that are not used by Linux to start an executable. Buildroot stopped supporting it because it can break executables. Try it only if saving a few bytes is critical.

Quick splashscreen display (1)

Often the first sign of life that you are showing!

- A good solution seems to be BusyBox `fbsplash`: See miscutils/fbsplash.c in BusyBox sources.
- Alternative: `fbv` `http://s-tech.elsat.net.pl/fbv/`
- However, `fbv` is slow: 878 ms on an Microchip AT91SAM9263 system!

Quick splashscreen display (2)

- To do it faster, you can dump the framebuffer contents:

```
fbv -d 1 /root/logo.bmp cp /dev/fb0 /root/logo.fb lzop -9 /root/logo.fb
```

- And then copy it back as early as possible in an initramfs:

```
lzopcat /root/logo.fb.lzo > /dev/fb0
```

Results on an Microchip AT91SAM9263 system:

	`fbv`	plain copy (`dd`)	`lzopcat`
Time	878 ms	54 ms	52.5 ms

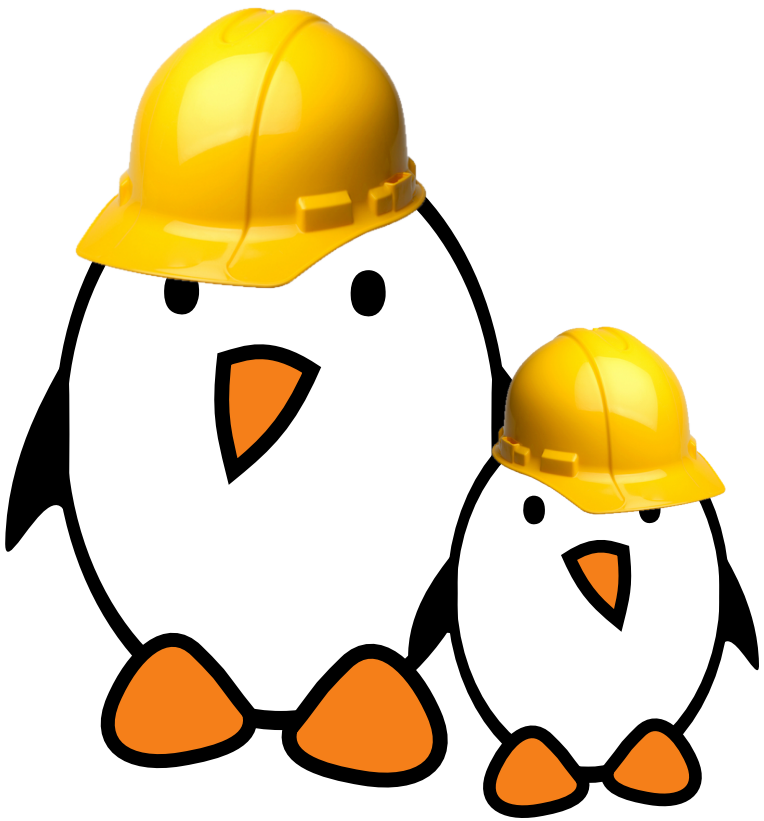
`https://bootlin.com/blog/super-fast-linux-splashscreen/` Note: *LZO* compression is the fastest in terms of decompression, and is supported by BusyBox.

Animated splashscreen

Still slow to read and write entire screens. Just draw useful pixels and even create an animation!

- Create a simple C program that just animates pixels and simple geometric shapes on the framebuffer!
- Example: `https://bootlin.com/pub/code/fb/anim.c` (Public Domain license). On a 400 MHz ARM9 system: starts drawing in 10 ms Size: 24 KB, compiled statically with Musl (2023 status).

Practical lab - Reducing time in init-scripts



- Regenerate the root filesystem with Buildroot
- Use bootchartd to measure boot time